Common Core: Phase 1 RTL Design

Lab R3: Arbitrated Memory

Interfaces, Arrays and a Round-Robin Arbiter

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 24, 2026

LAB OBJECTIVE Design a memory shared by two masters through a round-robin arbiter, connect it with a parameterized SystemVerilog interface with modports, choose packed and unpacked arrays deliberately, and prove the design with a small directed self-checking smoke testbench.

Duration	1.5 weeks (approximately 25–30 hours)
Prerequisites	Labs L0, R1 and R2
Designs	`mem_if` (interface), `arb_mem` (new)
Reused in	R4 (memory side of the cache), V2 (verification), P3 (area and timing), T-RTL2, T-DV1 and T-DV2 (`mem_if` drivers and agents)

Contents

1 Overview **3**

2 Design Specification **3**

2.1 Block Diagram . . . 3

2.2 Parameters . . . 4

2.3 The `mem_if` Interface . . . 4

2.4 Protocol . . . 4

2.5 Arbitration . . . 5

2.6 RTL Requirements . . . 5

3 Interfaces **5**

3.1 Interfaces and the Tools . . . 6

4 Synthesizable Arrays: Packed and Unpacked **7**

4.1 How to Choose . . . 7

4.2 Rules for Arrays in RTL . . . 7

5 The Round-Robin Arbiter **8**

5.1 Why Round Robin . . . 8

5.2 Two Masters . . . 8
5.3 Generalizing to N Masters . . . 8
5.4 Datapath Around the Arbiter . . . 9
5.5 A Conflict, Cycle by Cycle . . . 9

6 Smoke Testbench **10**
6.1 Structure . . . 10
6.2 Skeleton . . . 11
6.3 Directed Tests . . . 12
6.4 Rules . . . 13

7 Requirements / Tasks **13**

8 Repository and Flow **14**
8.1 Files Created in This Lab . . . 14
8.2 File Lists . . . 14
8.3 Makefile . . . 15
8.4 Commands . . . 15

9 Deliverables **16**

10 Assessment **16**

11 Review Questions **17**

12 References **17**

1 Overview

A processor core shares resources all the time: the instruction fetch unit and the load/store unit both access memory, and they ask at the same time. In this lab you build a small version of that situation: a memory with two master ports and a round-robin arbiter that decides who is served in each cycle.

The design is deliberately simple. The learning goals are the language features and the micro-architecture you need to build it cleanly:

- **Interfaces** bundle the signals of one port and define the direction of each signal per side (modports). The same interface, `mem_if`, is used again on both sides of the data cache in Lab R4 and by every testbench that talks to these blocks later.
- **Synthesizable static arrays**: an unpacked array holds the memory contents, packed arrays hold the control vectors of the arbiter. Choosing between them is a design decision with consequences for synthesis and for the code you can write.
- **A round-robin arbiter**: the smallest fair arbiter, and a building block that appears in every bus, interconnect and network-on-chip.

Where this lab sits. It builds on the coding rules and lint flow of Lab R1 and the simulation semantics and `<=` drive standard of Lab R2. Its output, `hw/ip/arb_mem`, is reused *in place* by Lab R4 as the main memory behind the cache, verified in depth in Lab V2 (virtual interfaces, `fork`/`join`, protocol assertions, random traffic), and synthesized for several address widths in Lab P3. Because so many later labs depend on it, the interface you define here must be right.

By the end of this lab you should be able to:

1. Define a parameterized interface with modports and use it as a module port in synthesizable RTL.
2. Explain the difference between packed and unpacked arrays, and choose the right one for a memory and for a control vector.
3. Design a round-robin arbiter for two masters, and explain how it generalizes to N masters.
4. Write a valid/ready slave whose `ready` depends combinationally on `valid` without creating a combinational loop.
5. Write a directed, self-checking smoke testbench that checks both data and arbitration order from a single `initial` block.
6. Compose file lists so that a later block can reuse the interface without copying it.

TEST TERM | Directed test

> A hand-written stimulus sequence with a known expected result, aimed at one feature ("after a single access by master 0, master 1 wins the next conflict"). Easy to debug, because you know what should happen in every cycle, but it only checks what its author thought of. Every `+TEST=` of this lab is a directed test; random traffic follows in Lab V2.

2 Design Specification

2.1 Block Diagram

Figure 1 shows `arb_mem` in its system: two masters, each connected through its own `mem_if` instance. The internals are detailed in Figure 2 (Section 5.4).

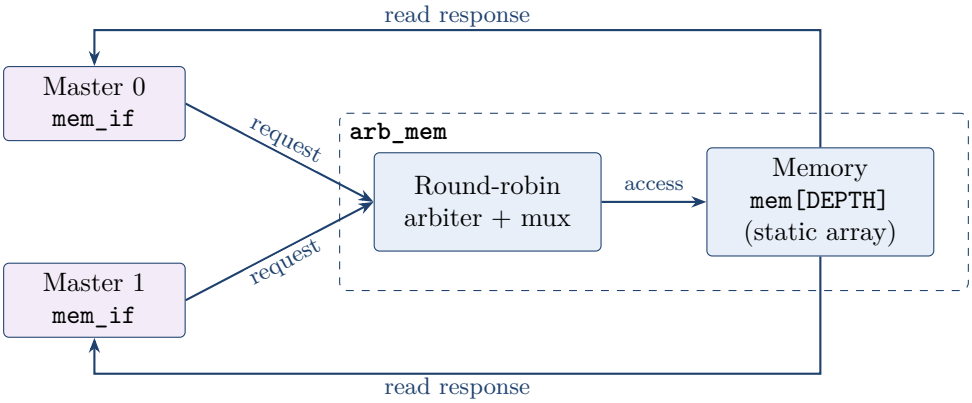


Figure 1: `arb_mem` and its two masters. Each master port is one `mem_if` instance.

2.2 Parameters

Parameter	Default	Meaning
`AW`	4	Address width; the memory has `DEPTH = 2**AW` words
`DW`	32	Data width

`DEPTH` is a `localparam` derived from `AW`; it is never set directly. The design must work for any `AW` ≥ 1; the smoke testbench runs `AW=2`, `4` and `6`. Lab R4 instantiates `arb_mem` with `AW=10`.

2.3 The `mem_if` Interface

Signal	Driven by	Width	Description
`req_valid`	master	1	A request is present
`req_ready`	slave	1	The request is accepted at this edge
`req_we`	master	1	1 = write, 0 = read
`req_addr`	master	`AW`	Word address
`req_wdata`	master	`DW`	Write data
`rsp_valid`	slave	1	Read data is valid
`rsp_rdata`	slave	`DW`	Read data

The interface also has the inputs `clk` and `rst_n`, and three modports: `master`, `slave` and `monitor` (all inputs).

2.4 Protocol

1. A request is *accepted* at a rising edge where `req_valid && req_ready`.
2. While `req_valid && !req_ready`, the master holds `req_valid`, `req_we`, `req_addr` and `req_wdata` stable.
3. `req_valid` must not depend on `req_ready`. `req_ready` may depend combinationally on `req_valid` (the arbiter needs it).
4. An accepted write updates the memory at the accepting edge. Writes have no response.

5. An accepted read produces `rsp_valid = 1` for exactly one cycle, exactly one cycle after acceptance, with the data of that address including every write accepted at an earlier edge.

Rule 3 is what prevents a combinational loop: if the master also computed `req_valid` from `req_ready`, the two signals would form a cycle through two blocks that neither designer can see alone. The rule is the same one AXI and TileLink use, and it is written down here because it cannot be checked by looking at one side only.

2.5 Arbitration

- At most one request (from either master) is accepted per cycle.
- If only one master requests, it is accepted in the same cycle.
- If both request, the master that was *not* granted most recently wins (round robin). A waiting master is therefore accepted after at most one cycle of waiting.
- After reset, master 0 has priority.

2.6 RTL Requirements

- Top module:

```
module arb_mem #(parameter int AW = 4, parameter int DW = 32) (
  input logic  clk, rst_n,
  mem_if.slave m0,
  mem_if.slave m1
);
```

- The memory is an unpacked static array: `logic [DW-1:0] mem [DEPTH]`. It is not reset; it is a datapath array (Lab R1, rule 6). Put it in its own `always_ff` block without a reset branch.
- Use at least one packed array for control (for example `logic [1:0] req, gnt;`) and explain in `doc/design.md` why it is packed and the memory is unpacked.
- `rsp_rdata` comes from a register, not directly from the memory array; `rsp_valid` is registered.
- The round-robin state is the only control register besides the response flags; it is reset.
- All Lab R1 coding rules apply; the design is lint-clean (`make lint`), and every waiver in `lint/arb_mem.vlt` has a justification comment.

NOTE | Synthesis.

The memory is built from flip-flops when synthesized, because no SRAM macro is available; that is why `DEPTH` is small. How area and timing grow with `AW` is measured in Lab P3.

3 Interfaces

Without interfaces, a port with seven signals must be declared in the master, in the slave, in every level of hierarchy between them and in the testbench — and all those lists must stay consistent. An interface declares the bundle once.

```
// hw/ip/arb_mem/rtl/mem_if.sv
interface mem_if #(parameter int AW = 4, parameter int DW = 32)
                  (input logic clk, input logic rst_n);
  logic          req_valid, req_ready, req_we;
  logic [AW-1:0] req_addr;
```

```
  logic [DW-1:0] req_wdata;
  logic          rsp_valid;
  logic [DW-1:0] rsp_rdata;

  modport master  (input  clk, rst_n, req_ready, rsp_valid, rsp_rdata,
                   output req_valid, req_we, req_addr, req_wdata);
  modport slave   (input  clk, rst_n, req_valid, req_we, req_addr, req_wdata,
                   output req_ready, rsp_valid, rsp_rdata);
  modport monitor (input  clk, rst_n, req_valid, req_ready, req_we,
                   req_addr, req_wdata, rsp_valid, rsp_rdata);

`ifndef SYNTHESIS
  // Protocol assertions go here. They are written in Lab V2.
`endif
endinterface
```

Key points:

- A **modport** defines the direction of each signal from one side's point of view. The tool reports an error if a module drives a signal its modport declares as an input. Use the modport in the port declaration (`mem_if.slave m0`), never the bare interface type, so the direction is part of the module's contract.
- The `monitor` modport has only inputs. Nothing in this lab uses it yet; testbench monitors in Labs V2, T-DV1 and T-DV2 do. Define it now so the interface does not change later.
- Interface parameters must match the DUT parameters; the testbench passes the same `AW` and `DW` to both. A mismatch is an elaboration error, which is the point: widths cannot silently disagree.
- **Protocol assertions belong in the interface**: every instance then checks its own port automatically, in this design and in any future design that uses it (the data cache of Lab R4 has two). In this lab you only leave the guarded placeholder; the assertions for the five protocol rules of Section 2.4 are written in Lab V2.
- An interface instance is a piece of hierarchy, not a variable. Passing a *handle* to an interface to a task (a `virtual interface`) is a testbench technique and is introduced in Lab V2. In this lab, the smoke testbench accesses the two interface instances directly by name.

3.1 Interfaces and the Tools

- **File order.** `mem_if.sv` is compiled before every module that uses it. It lives in `hw/ip/arb_mem/rtl/`, because it is part of the ports of `arb_mem`, and it gets its own file list `rtl/mem_if.f` so that Lab R4 can include the interface without including the memory.
- **Lint.** Verilator does not accept interface ports on the top-level module of a lint or simulation run. Lint therefore runs on a thin wrapper, `lint/arb_mem_lint_top.sv`, that instantiates two `mem_if` and `arb_mem` and exposes the interface signals as ordinary ports (see Section 8). The wrapper is not part of the RTL file list.
- **Synthesis.** sv2v flattens interfaces into ordinary ports (`m0_req_valid`, ...); you will see these names in the netlists of Lab P3.

SYNOPSYS CLOUD

SpyGlass (`make lint LINT_TOOL=spyglass`, the cloud default) and VC SpyGlass (`LINT_TOOL=vcst`) lint the same wrapper through `LINT_TOP`/`LINT_FLIST`, so one Makefile serves all three lint tools. Their waivers go in `lint/arb_mem.awl` (SpyGlass) and `lint/arb_mem.vcst.tcl` (VC SpyGlass), each with a justification like the `.vlt` file. Expect `W240` ("input declared but not read") for `m0.clk`/`m0.rst_n`: the modport carries the clock for the interface assertions of Lab V2, while `arb_mem` uses its own `clk`/`rst_n`

ports — a justified waiver, not a code change. Fusion Compiler (`SYN_TOOL=fc`, Lab P3) reads interfaces directly; sv2v is not needed.

4 Synthesizable Arrays: Packed and Unpacked

SystemVerilog has five kinds of arrays. Only the two static kinds describe hardware; the other three (dynamic arrays, queues and associative arrays) exist only in simulation. You have already used a queue as the reference model in the Lab R2 smoke testbench; the full set of testbench arrays and when to use each is taught in Lab V2. This lab is about the two synthesizable kinds.

Kind	Declaration	Properties
Packed	`logic [7:0] b;`	Contiguous bits; usable as one number in arithmetic, comparison, reduction and shifts
Packed, multi-dim.	`logic [3:0][7:0] w;`	Four bytes that are also one 32-bit vector: `w[2]` is a byte, `w` is a number
Unpacked, static	`logic [7:0] m [16];`	Fixed number of separate elements; size known at compile time; one element accessed at a time
Mixed	`logic [DW-1:0] mem [DEPTH];`	Unpacked array of packed words: the usual form of a memory

4.1 How to Choose

- **Packed** when the bits belong together as one value: a request vector you want to test with `|req` or `$onehot0(gnt)`, a vector you do arithmetic on (the `req & ~(req - 1)` trick below), a bus. Packed arrays can be assigned, compared and sliced as a whole.
- **Unpacked** when the elements are separate storage locations that are addressed one at a time: a memory, a register file, a table of tags. Synthesis tools recognize the pattern "unpacked array, one write port, indexed read" as a memory and can map it to an SRAM macro when one exists (Lab P3).
- A memory declared as one big packed vector (`logic [DEPTH*DW-1:0]`) simulates, but every access becomes a variable part-select, synthesis can no longer infer a memory, and the code is unreadable. Do not do it.

4.2 Rules for Arrays in RTL

- Size unpacked arrays with the C-style form `[DEPTH]` (index 0 to `DEPTH-1`) or `[0:DEPTH-1]`; be consistent.
- Indexing with an address of exactly `$clog2(DEPTH)` bits cannot go out of range when `DEPTH` is a power of two. If it is not, an out-of-range read returns X in simulation and an arbitrary value in hardware: guard it or make it impossible.
- Use indexed part-selects (`v[base +: width]`) for fields whose position depends on a parameter; they are synthesizable because the width is constant.
- Do not reset memory arrays. A reset loop over `DEPTH` entries turns every storage bit into a resettable flop and prevents memory inference (Lab R1, rule 6).
- A `for` loop over an array describes parallel hardware, not a sequence in time (Lab R1, rule 10).

5 The Round-Robin Arbiter

5.1 Why Round Robin

A **fixed-priority** arbiter always grants the lowest-numbered requester. It is the cheapest arbiter, but a master that requests in every cycle starves all masters below it: they may never be served. A **round-robin** arbiter rotates the priority: the master granted most recently gets the *lowest* priority in the next arbitration. For N masters that all keep requesting, each one waits at most $N-1$ grants. For two masters this is exactly the specification's "at most one cycle of waiting".

5.2 Two Masters

The state is one bit that says who has priority on the next conflict. It changes only when a grant happens:

`req[1:0]`	`prio` (0 = master 0 first)	`gnt[1:0]`	`prio_next`	Comment
00	x	00	`prio`	idle: state unchanged
01	x	01	1	master 0 alone
10	x	10	0	master 1 alone
11	0	01	1	conflict, master 0 wins
11	1	10	0	conflict, master 1 wins

Note that a single request also moves the priority: the specification says "the master that was not granted most recently wins", not "the master that did not win the last conflict". Your smoke testbench checks exactly this difference (tests `rr_after_m0` and `rr_after_m1` in Section 6.3).

5.3 Generalizing to N Masters

A clean way to build an N-way round-robin arbiter is a *masked priority* scheme: two fixed-priority arbiters, one on the requests above the last winner and one on all requests. The packed-array arithmetic below is the reason the control vectors are packed.

```
logic [N-1:0] req, gnt, mask_q, req_m, gnt_m, gnt_u;

// Fixed priority (lowest index wins): isolate the lowest set bit.
assign req_m = req & mask_q;          // requests above the last winner
assign gnt_m = req_m & ~(req_m - 1'b1);
assign gnt_u = req   & ~(req   - 1'b1);
assign gnt   = (|req_m) ? gnt_m : gnt_u;

// After a grant to index k, only indices k+1..N-1 stay unmasked.
always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n)    mask_q <= '1;        // after reset: index 0 first
  else if (|gnt) mask_q <= ~((gnt << 1) - 1'b1);
end
```

Check it for $N = 2$: after a grant to master 0 the mask is `10`, so master 1 wins the next conflict; after a grant to master 1 the mask is `00`, the masked arbiter is empty, and the unmasked one gives master 0. You may implement the two-master arbiter with the one-bit table above or with this masked form (with `N=2`); explain the choice in `doc/design.md`. A generic `rr_arb #(N)` module in its own file is welcome but not required.

5.4 Datapath Around the Arbiter

Figure 2 shows the complete micro-architecture: one control register (`prio_q`), one response-flag register (`rsp_sel_q`), the read-data register and the memory array. Every arrow that crosses the dashed boundary is a signal of the `mem_if.slave` modport of `m0` (left) or `m1` (right).

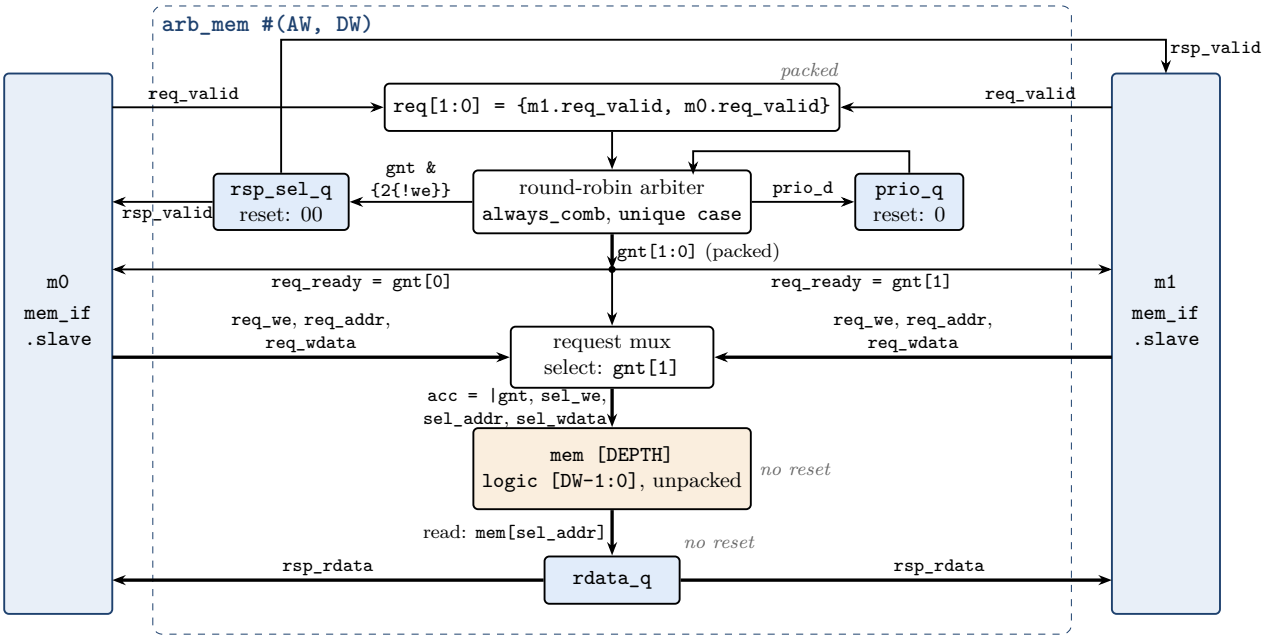
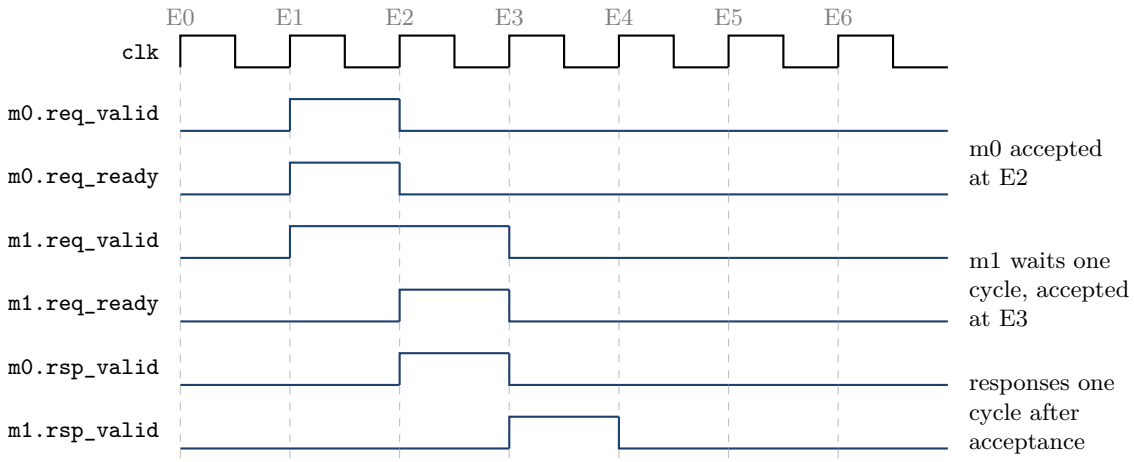


Figure 2: Inside `arb_mem`: blue boxes are registers, white boxes combinational logic, the orange box the memory array. `m0` and `m1` are the two `mem_if` instances seen through the `slave` modport (the `master` modport is its mirror image, `monitor` has only inputs). `req_ready` depends combinationally on `req_valid` through the arbiter; nothing depends on `req_ready` inside the block.

- **Request vector.** `req = {m1.req_valid, m0.req_valid}`.
- **Ready.** `m0.req_ready = gnt[0]`, `m1.req_ready = gnt[1]`. This is the combinational `valid` → `ready` path that protocol rule 3 allows. Should `req_ready` be high when nobody requests? It does not matter for the protocol; choose, and document why.
- **Request mux.** `we`, `addr` and `wdata` are selected by `gnt`. Use `unique case` or a conditional on `gnt[1]`; do not let a missing default create a latch (Lab R1, rule 1).
- **Memory.** On an accepted write, `mem[addr] <= wdata`. On an accepted read, `rdata_q <= mem[addr]`. Since at most one access happens per cycle, a read accepted at an edge sees every write accepted at earlier edges.
- **Response routing.** Register which master the read belongs to (for example `rsp_sel_q <= gnt & {2{!we}}`, a packed vector) and drive `rsp_valid` of each port from its bit. `rsp_rdata` may go to both ports; only the port with `rsp_valid` looks at it.

5.5 A Conflict, Cycle by Cycle

Both masters issue a read in cycle 1. Master 0 has priority after reset, so it is accepted at edge 2; master 1 holds its request (protocol rule 2) and is accepted at edge 3. Each response appears in the cycle after its acceptance. Your `doc/design.md` must contain this diagram as WaveDrom source (Section 9), extended with a write.



6 Smoke Testbench

The designer's smoke testbench proves that the block basically works, for every parameter set, in a few seconds. It is directed and fully self-checking, and it is deliberately limited: one `initial` block drives both masters in a fixed sequence. Running the two masters as independent parallel processes, random traffic, virtual interfaces, statistics and protocol assertions are Lab V2; do not build them here.

TEST TERM | Random test

Stimulus drawn from `$urandom` with a printed seed, thousands of transactions, checked by a reference model and a scoreboard. It finds the conflicts nobody thought of; the seed makes a failure reproducible. It needs a testbench whose two masters run independently, which is why it belongs to Lab V2 and not to the smoke test.

6.1 Structure

Part	Responsibility
Interfaces and DUT	Two `mem_if` instances, `m0_if` and `m1_if`, with the testbench parameters; `arb_mem` connected to both
Reference model	Static unpacked arrays `model[DEPTH]` and `written[DEPTH]`, updated in the order the DUT *accepts* requests
`drive()`	Drives one master's request signals with `<=` (the Lab R2 standard)
`step()`	Waits for one rising edge and checks everything that happened at that edge: responses, acceptances, at most one acceptance; updates the model
Request tasks	`write(id, ...)`, `read(id, ...)`, `conflict(...)`: hold a request until accepted, then release it
Watchdog	A second `initial` block that fails the test after a fixed number of cycles
Report	Prints the configuration first; ends with exactly one `TEST PASSED` or `TEST FAILED`

Figure 3 shows how the parts connect. The testbench has no parallel processes besides the watchdog: the single test process both drives (through `drive()`) and checks (through `step()`), and the model is updated only from what `step()` observed at the edge.

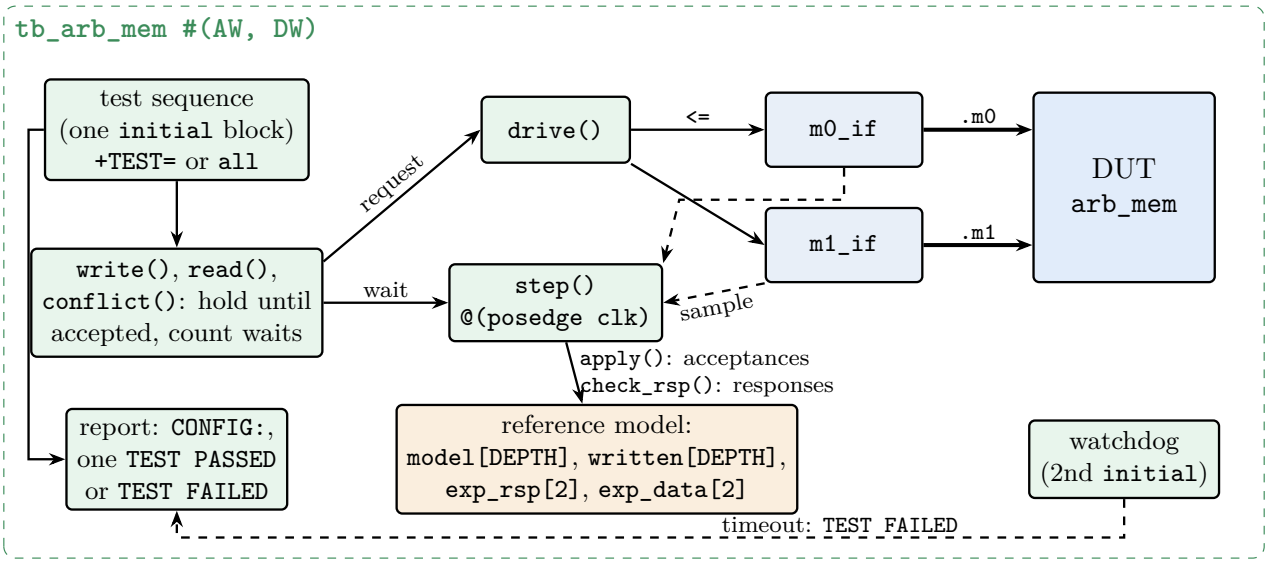


Figure 3: Structure of the R3 smoke testbench. Green: testbench code; dashed arrows: sampling at the rising edge (before the NBA region). The reference model is updated in the order the DUT accepted the requests, never in the order the test issued them.

6.2 Skeleton

```
module tb_arb_mem #(parameter int AW = 4, parameter int DW = 32);
  localparam int DEPTH = 2**AW;
  logic clk = 1'b0, rst_n;
  initial forever #5 clk = ~clk;

  mem_if #(.AW(AW), .DW(DW)) m0_if (.clk, .rst_n);
  mem_if #(.AW(AW), .DW(DW)) m1_if (.clk, .rst_n);
  arb_mem #(.AW(AW), .DW(DW)) dut (.clk, .rst_n, .m0(m0_if), .m1(m1_if));

  logic [DW-1:0] model [DEPTH];   // reference model: static arrays,
  bit            written [DEPTH]; // updated in DUT acceptance order
  bit            exp_rsp [2];     // read accepted at the previous edge
  logic [DW-1:0] exp_data [2];
  bit            acc [2];         // accepted at the edge just passed
  int unsigned   checks, errors;

  // Wait for an edge; sample what happened AT that edge (values before
  // the NBA region of this time step, see Lab R2), check, update model.
  task automatic step();
    @(posedge clk);
    acc[0] = m0_if.req_valid && m0_if.req_ready;
    acc[1] = m1_if.req_valid && m1_if.req_ready;
    check_rsp(0, m0_if.rsp_valid, m0_if.rsp_rdata);  // uses exp_rsp[0]
    check_rsp(1, m1_if.rsp_valid, m1_if.rsp_rdata);
    if (acc[0] && acc[1]) fail("two acceptances in one cycle");
    if (acc[0]) apply(0, m0_if.req_we, m0_if.req_addr, m0_if.req_wdata);
    if (acc[1]) apply(1, m1_if.req_we, m1_if.req_addr, m1_if.req_wdata);
  endtask

  // Selecting the port with an if is clumsy; Lab V2 replaces it
  // with virtual interface handles.
  task automatic drive(int id, bit valid, bit we = 1'b0,
                       logic [AW-1:0] addr = '0, logic [DW-1:0] wdata = '0);
    if (id == 0) begin
```

```
      m0_if.req_valid <= valid; m0_if.req_we <= we;
      m0_if.req_addr  <= addr;  m0_if.req_wdata <= wdata;
    end else begin
      m1_if.req_valid <= valid; m1_if.req_we <= we;
      m1_if.req_addr  <= addr;  m1_if.req_wdata <= wdata;
    end
  endtask

  task automatic write(int id, logic [AW-1:0] addr, logic [DW-1:0] wdata);
    drive(id, 1'b1, 1'b1, addr, wdata);
    do step(); while (!acc[id]);      // hold until accepted (rule 2)
    drive(id, 1'b0);
  endtask
  // read(), conflict(), check_rsp(), apply(), fail(), tests, watchdog ...
endmodule
```

`check_rsp(id, ...)` expects `rsp_valid` exactly when `exp_rsp[id]` is set and compares the data when the address was written (`written[addr]`); otherwise the data is X and is not compared, but the read is still counted. Then it clears `exp_rsp[id]`. `apply()` updates the model on a write, or sets `exp_rsp` and `exp_data` on a read. Because `step()` checks responses before it applies the acceptances of the same edge, a response is matched to the read accepted at the previous edge: exactly protocol rule 5.

6.3 Directed Tests

`+TEST=` selects one test; the default `all` runs them in order. Every test also checks that each request is accepted in the cycle the specification says (record the cycle count in `write()` / `read()` and compare with the expected wait).

TEST TERM | Corner-case test

A directed test at the edge of the specification, where most bugs live. Here: both masters requesting in the same cycle directly after reset (`conflict`), the priority update after a *single* access (`rr_after_m0`/`_m1`), and a write and a read of the same address in the same cycle (`cross_rw`). A pass proves the rule for exactly the situation tested.

+TEST=	Scenario and expected result	What a pass shows
`reset`	After reset: no `rsp_valid` on either port for several cycles without requests	Response flags are reset; no spurious response (P5)
`single_m0`	Master 0 writes every address, then reads every address back; master 1 idle. Every request accepted without waiting	Whole address range, data path, lone requester accepted at once (A2)
`single_m1`	The same for master 1	The same through the second port and mux input
`back_to_back`	Master 0 issues reads in consecutive cycles (keeps `req_valid` high, changes the address after each acceptance): one response per cycle	Full throughput; responses exactly one cycle after each read (P5)
`conflict`	Directly after reset, both masters request in the same cycle: master 0 accepted first, master 1 one cycle later	Reset priority (A4), one acceptance per cycle (A1), the loser holds its request (P2)
`rr_after_m0`	A single access from master 0, then a conflict: master 1 must win	A single grant also moves the priority (A3)
`rr_after_m1`	A single access from master 1, then a conflict: master 0 must win	The same in the other direction
`alternate`	Both masters keep requesting for 8 cycles (new request after each acceptance): grants alternate 0, 1, 0, 1, ...	Fairness under permanent conflict: no master waits more than one cycle
`cross_rw`	In one cycle, master 0 writes address A and master 1 reads A; repeated with the priority on the other side. The read returns the old or the new value according to the grant order — the model handles it because it follows acceptance order	Read-after-write ordering across masters (P4, P5)

6.4 Rules

- All DUT inputs are driven with `<=`, synchronous to the rising edge (Lab R2). Initialize every request signal to 0 before reset is released.
- The model is updated only in `step()`, in the order the DUT accepts requests — never in the order the test issues them. In `cross_rw` this is the only correct choice.
- The testbench prints `AW`, `DW` and `TEST` at the start, the number of checks and errors at the end, and exactly one `TEST PASSED` / `TEST FAILED` line.
- The watchdog ends the simulation with `TEST FAILED` if it hangs (for example, a master that is never granted).
- No assertions, `fork`, virtual interfaces or random stimulus: those are Lab V2.

7 Requirements / Tasks

1. Create the branch `lab/r3-arb-mem` and the directory `hw/ip/arb_mem/` with the layout of Section 8.
2. Write `doc/spec.md`: parameters, `mem_if` signal table, the five protocol rules and the arbitration rules, in your own words.
3. Write `rtl/mem_if.sv` with the three modports and the guarded placeholder for protocol assertions.
4. Write `rtl/arb_mem.sv` according to Section 2.6: round-robin arbiter, request mux, unpacked memory array, registered read data and response routing.

5. Write the file lists `rtl/mem_if.f` and `rtl/arb_mem.f` (nested `-f`), the lint wrapper and `lint/arb_mem.vlt`; `make lint` is clean.
6. Write `tb/tb_arb_mem.sv` with all tests of Section 6.3; `make sim` passes for `PARAMS="AW=2 DW=32"`, `"AW=4 DW=32"` and `"AW=6 DW=32"`, and for `DW=8`.
7. Break the arbiter on purpose in a scratch copy (for example, the priority never changes) and confirm that the smoke testbench fails, and in which test. Do not commit the broken copy; record the result in `doc/design.md`.
8. Write `doc/design.md`: block diagram, arbiter choice and state, the packed/unpacked justification, the `req_ready`-without-request choice, and the WaveDrom timing diagram of a conflict with a write and a read.
9. Confirm that `make -C $REPO_ROOT check-all` still passes (Labs L0–R2 are not broken), update `README.md`, open the merge request and tag `r3-done` after review.

TEST TERM | Negative / error-injection test

A bug is inserted on purpose to prove that the checker can fail. A testbench that has never failed proves nothing: it may not be checking at all. With the priority stuck at master 0, the first failing test must be `rr_after_m0`; if every test still passes, the testbench does not check arbitration.

8 Repository and Flow

8.1 Files Created in This Lab

```
hw/ip/arb_mem/
|-- README.md                 how to run, results, known limitations
|-- Makefile
|-- doc/
|   |-- spec.md               interface, protocol, arbitration
|   `-- design.md             block diagram, arbiter, WaveDrom conflict
|-- rtl/
|   |-- mem_if.sv             interface (part of the ports, so it lives here)
|   |-- mem_if.f              interface only (reused by hw/ip/dcache in R4)
|   |-- arb_mem.sv
|   `-- arb_mem.f             -f mem_if.f + arb_mem.sv
|-- lint/
|   |-- arb_mem.vlt           waivers, each with a justification
|   |-- arb_mem_lint_top.sv   wrapper with plain ports (lint only)
|   `-- lint.f                -f ../rtl/arb_mem.f + the wrapper
`-- tb/
    |-- tb_arb_mem.sv         directed smoke testbench
    `-- tb.f                  -f ../rtl/arb_mem.f + tb_arb_mem.sv
```

8.2 File Lists

```
// hw/ip/arb_mem/rtl/mem_if.f
${REPO_ROOT}/hw/ip/arb_mem/rtl/mem_if.sv

// hw/ip/arb_mem/rtl/arb_mem.f
-f ${REPO_ROOT}/hw/ip/arb_mem/rtl/mem_if.f
${REPO_ROOT}/hw/ip/arb_mem/rtl/arb_mem.sv

// hw/ip/arb_mem/tb/tb.f
```

```
-f ${REPO_ROOT}/hw/ip/arb_mem/rtl/arb_mem.f
${REPO_ROOT}/hw/ip/arb_mem/tb/tb_arb_mem.sv
```

The interface is listed before its users (packages and interfaces before users, Lab L0). Splitting `mem_if.f` from `arb_mem.f` costs one small file now and lets the data cache in Lab R4 depend on the interface alone.

8.3 Makefile

```
# hw/ip/arb_mem/Makefile
IP          := arb_mem
DUT_TOP     := arb_mem
TB_TOP      ?= tb_arb_mem
RTL_FLIST   := rtl/arb_mem.f
TB_FLIST    ?= tb/tb.f
# Verilator: no interface ports on the top module -> lint a wrapper
LINT_TOP    := arb_mem_lint_top
LINT_FLIST := lint/lint.f
PARAMS      ?= AW=4 DW=32
# the smoke testbench runs every test in order by default
TEST        ?= all
include $(REPO_ROOT)/flow/mk/common.mk
include $(REPO_ROOT)/flow/mk/lint.mk
include $(REPO_ROOT)/flow/mk/sim_verilator.mk
```

`LINT_TOP` and `LINT_FLIST` default to `DUT_TOP` and `RTL_FLIST`. If the `lint.mk` you wrote in Lab L0 does not have them yet, add them generically (with those defaults) and confirm with `check-all` that the earlier IPs still lint unchanged. Do not add IP-specific code to `flow/mk/`.

8.4 Commands

```
source env.sh
make lint IP=arb_mem                          # from the repository root
cd hw/ip/arb_mem
make sim                                      # TEST=all, default PARAMS
make sim TEST=alternate WAVES=1 && make waves
make sim PARAMS="AW=2 DW=32"                  # build/arb_mem/AW-2_DW-32/sim/
make sim PARAMS="AW=6 DW=8"
make -C $REPO_ROOT check-all                  # earlier labs still pass
```

SYNOPSYS CLOUD

The commands are the same; `env.sh` selects `SIM=vcs` and `LINT_TOOL=spyglass`. `make sim WAVES=1` dumps FSDB and `make waves` opens Verdi: the two interface instances `m0_if` and `m1_if` appear in the hierarchy like modules, and their signals can be dragged into the waveform as one group per port — the fastest way to see a conflict. VCS is four-state: a read of a never-written address returns X (the testbench counts but does not compare it), where Verilator returns 0. Verilator 5.013 on the cloud reports `INITIALDLY` for the `<=` drives in the testbench tasks and executes them as blocking assignments, which breaks the drive standard of Lab R2: run the smoke test with `SIM=vcs` there (`make lint LINT_TOOL=verilator` still works).

No new make targets or variables are introduced by this lab except the optional generic `LINT_TOP`/`LINT_FLIST`. Synthesis of `arb_mem` is done in Lab P3 with the targets of Labs P1 and P2.

9 Deliverables

The merge request from `lab/r3-arb-mem` contains:

- The RTL, file lists, lint wrapper and waivers, smoke testbench and Makefile of Section 8; `make lint sim` passes for the three `AW` values.
- `doc/spec.md`: the interface and the protocol as the contract other blocks rely on.
- `doc/design.md`:
  - block diagram and arbiter description (state, update rule, one-bit or masked form and why);
  - why the memory is an unpacked array and `req` / `gnt` are packed;
  - a WaveDrom timing diagram of a conflict: both masters request in the same cycle (one write, one read), one waits, then the read response;
  - the result of the broken-arbiter experiment.
- `README.md`: how to run every target, the result of each test for each `PARAMS` set, known limitations (for example: no byte enables, single-cycle memory, two masters only).

A WaveDrom source has this form (render it with the WaveDrom editor or the Markdown plugin of your Git server):

```
{ "signal": [
  { "name": "clk",          "wave": "p......" },
  { "name": "m0.req_valid", "wave": "010...." },
  { "name": "m0.req_ready", "wave": "010...." },
  { "name": "m1.req_valid", "wave": "01.0..." },
  { "name": "m1.req_ready", "wave": "0.10..." },
  { "name": "m0.rsp_valid", "wave": "0.10..." },
  { "name": "m1.rsp_valid", "wave": "0..10.." }
]}
```

(The example above is only a format sample; draw your own design's behavior, which may differ in when `ready` rises.)

10 Assessment

Criterion	Weight
Functional correctness for all `PARAMS` sets (smoke tests pass)	25%
Interface design: modports, parameters, file-list split	15%
Arbiter: fairness, priority update, combinational ready without loops	20%
RTL quality: coding rules, array choices, lint clean, justified waivers	15%
Smoke testbench: acceptance-order model, cycle checks, termination	15%
Documentation: spec, design notes, WaveDrom diagram, README	10%
Total	100%

11 Review Questions

1. Why is the memory an unpacked array and `gnt` a packed array? What would change if you swapped them?
2. Why must `req_valid` not depend on `req_ready`? What happens if both depend on each other combinationally?
3. What does a modport add over using the bare interface type as a port? Give an error it catches.
4. Why do protocol assertions belong in the interface rather than in `arb_mem`? Which later block benefits automatically?
5. With a fixed-priority arbiter, which test of your smoke testbench would fail, and which master would starve?
6. Show that `req & ~(req - 1)` isolates the lowest set bit. Why does this only work on a packed vector?
7. A read of address A and a write of A arrive from different masters in the same cycle. Which value does the read return, and why does your model get it right?
8. Why is `rsp_rdata` taken from a register and not directly from `mem[addr]`?
9. Why does `rtl/mem_if.f` exist separately from `rtl/arb_mem.f`?
10. What does the synthesized memory consist of, and why would a real chip use an SRAM macro instead? (You will measure it in Lab P3.)

12 References

- IEEE Std 1800-2017: Clause 7 (aggregate data types: packed and unpacked arrays), Clause 25 (interfaces and modports).
- *SystemVerilog for Design*, Sutherland, Davidmann and Flake: chapters on arrays and interfaces.
- lowRISC Verilog/SystemVerilog style guide (Lab R1).
- OpenTitan `prim_arbiter_ppc` and `prim_arbiter_tree`: production round-robin arbiters with the masked-priority scheme.
- PULP `common_cells`: `rr_arb_tree.sv`.
- WaveDrom: digital timing diagram rendering engine, https://wavedrom.com.